

Relational database vs. NoSQL database

Cassandra is a NoSQL database. The main objective of a NoSQL database is to have the following three things:

- Simplicity of design
- Horizontal scaling
- High availability

NoSQL is faster than relational database management system because it uses different data structure compared to relational databases.

Cassandra data structure is faster than relational database structure. NoSQL databases are mainly used in Bigdata and real time web applications.

Differences between NoSQL and Relational database

NoSQL Database	Relational Database
NoSQL Database supports a very simple query language.	Relational Database supports a powerful query language.
NoSQL Database has no fixed schema.	Relational Database has a fixed schema.
NoSQL Database is only eventually consistent.	Relational Database follows acid properties. (Atomicity, Consistency, Isolation, and Durability)
NoSQL databases don't support transactions (support only simple transactions).	Relational Database supports transactions (also complex transactions with joins).
NoSQL Database is used to handle data coming in high velocity.	Relational Database is used to handle data coming in low velocity.
The NoSQL's data arrive from many locations.	Data in relational database arrive from one or few locations.
NoSQL database can manage structured, unstructured and semi-structured data.	Relational database manages only structured data.
NoSQL databases have no single point of failure.	Relational databases have a single point of failure with failover.
NoSQL databases can handle big data or data in a very high volume .	NoSQL databases are used to handle moderate volume of data.

NoSQL has decentralized structure.	Relational database has centralized structure.
NoSQL database gives both read and write scalability.	Relational database gives read scalability only.
NoSQL database is deployed in horizontal fashion.	Relation database is deployed in vertical fashion.

MongoDB:-

MongoDB tutorial provides basic and advanced concepts of SQL.

MongoDB is a No SQL database. It is an open-source, cross-platform, document-oriented database written in C++.

Our MongoDB tutorial includes all topics of MongoDB database such as insert documents, update documents, delete documents, query documents, projection, sort() and limit() methods, create a collection, drop collection, etc. There are also given MongoDB interview questions to help you better understand the MongoDB database.

What is MongoDB

MongoDB is an open-source document database that provides high performance, high availability, and automatic scaling.

In simple words, you can say that - Mongo DB is a document-oriented database. It is an open source product, developed and supported by a company named 10gen.

MongoDB is available under General Public license for free, and it is also available under Commercial license from the manufacturer.

The manufacturing company 10gen has defined MongoDB as:

"MongoDB is a scalable, open source, high performance, document-oriented database." - 10gen

MongoDB was designed to work with commodity servers. Now it is used by the company of all sizes, across all industry.

History of MongoDB

The initial development of MongoDB began in 2007 when the company was building a platform as a service similar to window azure.

MongoDB was developed by a NewYork based organization named 10gen which is now known as MongoDB Inc. It was initially developed as a PAAS (Platform as a Service). Later in 2009, it is introduced in the market as an open source database server that was maintained and supported by MongoDB Inc.

The first ready production of MongoDB has been considered from version 1.4 which was released in March 2010.

MongoDB 2.4.9 was the latest and stable version which was released on January 10, 2014.

Purpose of Building MongoDB

It may be a very genuine question that - "what was the need of MongoDB although there were many databases in action?"

There is a simple answer:

All the modern applications require big data, fast features development, flexible deployment, and the older database systems not competent enough, so the MongoDB was needed.

The primary purpose of building MongoDB is:

- Scalability
- Performance
- High Availability
- Scaling from single server deployments to large, complex multi-site architectures.
- Key points of MongoDB
- Develop Faster
- Deploy Easier
- Scale Bigger

First of all, we should know what is document oriented database?

Example of Document-Oriented Database

MongoDB is a document-oriented database. It is a key feature of MongoDB. It offers a document-oriented storage. It is very simple you can program it easily.

MongoDB stores data as documents, so it is known as document-oriented database.

1. FirstName = "John",
2. Address = "Detroit",
3. Spouse = [{Name: "Angela"}].
4. FirstName = "John",
5. Address = "Wick"

There are two different documents (separated by ".").

Storing data in this manner is called as document-oriented database.

Mongo DB falls into a class of databases that calls Document Oriented Databases. There is also a broad category of database known as **No SQL Databases**.

Features of MongoDB

These are some important features of MongoDB:

1. Support ad hoc queries

In MongoDB, you can search by field, range query and it also supports regular expression searches.

2. Indexing

You can index any field in a document.

3. Replication

MongoDB supports Master Slave replication.

A master can perform Reads and Writes and a Slave copies data from the master and can only be used for reads or back up (not writes)

4. Duplication of data

MongoDB can run over multiple servers. The data is duplicated to keep the system up and also keep its running condition in case of hardware failure.

5. Load balancing

It has an automatic load balancing configuration because of data placed in shards.

6. Supports map reduce and aggregation tools.

7. Uses **JavaScript** instead of Procedures.

8. It is a schema-less database written in **C++**.

9. Provides high performance.

10. Stores files of any size easily without complicating your stack.

11. Easy to administer in the case of failures.

12. It also supports:

- JSON data model with dynamic schemas
- Auto-sharding for horizontal scalability
- Built in replication for high availability

- Now a day many companies using MongoDB to create new types of applications, improve performance and availability.

Cassandra: -

Cassandra tutorial provides basic and advanced concepts of Cassandra. Our Cassandra tutorial is designed for beginners and professionals both.

Cassandra is a NoSQL database which is distributed and scalable. It is provided by Apache.

Our Cassandra tutorial includes all topics of Cassandra such as features, architecture, Relational vs NoSQL, Cassandra vs HBase, installation, Keyspace, Table, Views, Cassandra Query Language, Security etc.

What is Cassandra?

Apache Cassandra is highly scalable, high performance, distributed NoSQL database. Cassandra is designed to handle huge amount of data across many commodity servers, providing high availability without a single point of failure.

Cassandra has a distributed architecture which is capable to handle a huge amount of data. Data is placed on different machines with more than one replication factor to attain a high availability without a single point of failure.

NoSQL database is Non-relational database. It is also called Not Only SQL. It is a database that provides a mechanism to store and retrieve data other than the tabular relations used in relational databases. These databases are schema-free, support easy replication, have simple API, eventually consistent, and can handle huge amounts of data.

Important Points of Cassandra

- Cassandra is a column-oriented database.
- Cassandra is scalable, consistent, and fault-tolerant.
- Cassandra's distribution design is based on Amazon's Dynamo and its data model on Google's Bigtable.
- Cassandra is created at Facebook. It is totally different from relational database management systems.

- Cassandra follows a Dynamo-style replication model with no single point of failure, but adds a more powerful "column family" data model.
- Cassandra is being used by some of the biggest companies like Facebook, Twitter, Cisco, Rackspace, ebay, Twitter, Netflix, and more.

Cassandra History:-

- Cassandra was developed at Facebook by Avinash Lakshman and Prashant Malik.
- It was developed for Facebook inbox search feature.
- It was open sourced by Facebook in July 2008.
- It was accepted by Apache Incubator in March 2009.
- Cassandra is a top level project of Apache since February 2010.
- The latest version of Apache Cassandra is 3.2.1.

HBASE: -

- HBase is a database for storing and retrieving data with random access. In other words, we can write data as per our requirements and read it back again as needed.
- HBase works well with both structured and semi-structured data.
- This means storing structured data like relational tables and semi-structured data like tweets or log files together is possible.
- If the data is not large, HBase can also handle unstructured data.

Features of HBase:-

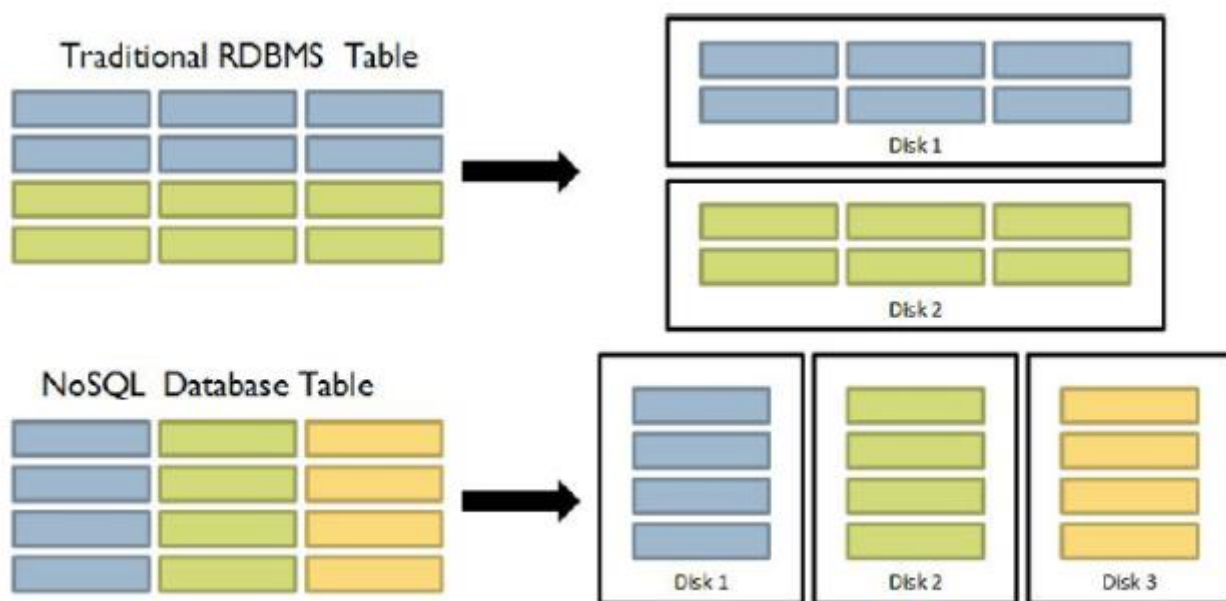
- Horizontally scalable: You can add any number of columns anytime.
- Automatic Failover: Automatic failover is a resource that allows a system administrator to automatically switch data handling to a standby system in the event of system compromise
- Integrations with Map/Reduce framework: All the commands and java codes internally implement Map/ Reduce to do the task and it is built over Hadoop Distributed File System.
- sparse, distributed, persistent, multidimensional sorted map, which is indexed by rowkey, column key, and timestamp.

- Often referred as a key value store or column family-oriented database, or storing versioned maps of maps.
- fundamentally, it's a platform for storing and retrieving data with random access.
- It doesn't care about datatypes(storing an integer in one row and a string in another for the same column).
- It doesn't enforce relationships within your data.
- It is designed to run on a cluster of computers, built using commodity hardware.

RDBMS vs HBase

There differences between RDBMS and HBase are given below:

- Schema/Database in RDBMS can be compared to namespace in Hbase.
- A table in RDBMS can be compared to column family in Hbase.
- A record (after table joins) in RDBMS can be compared to a record in Hbase.
- A collection of tables in RDBMS can be compared to a table in Hbase..



Challenges of NoSQL:

Back-ups in Application Consistent: Determining the sequence of changes across replicas, which is critical in selecting which values should compose a snapshot, is a typical difficulty in quorum-based replication systems. For example, determining a rigorous ordering between two write requests to the same database object that arrived at two distinct nodes at the same time is challenging. As a result of the absence of ordering, determining the most recent value of a database item at any given time is difficult.

Maturity: NoSQL proponents would agree that increasing demands will inevitably lead to obsolescence, but the RDBMS maturity cycle appears to be more secure. RDBMS is more reliable and functional since it has been

around for decades. Many NoSQL alternatives are still in the pre-production stage and do not yet have all of their essential features implemented.

Database and node failures during backups and restores: Node failures are common in NoSQL databases since they are designed to grow to hundreds of nodes. As a result, any backup method must be able to account for data collection failures from down nodes and their influence on quorum consistency. On the other hand, restorations must take into consideration failed cluster nodes and modify data re population correspondingly.

Analytics and business intelligence: NoSQL was created to fulfill the needs of Web 2.0 applications, and as a result, all of its characteristics are geared toward that goal. Other commercial systems, on the other hand, necessitate moving beyond the insert-read-update-delete cycle. Even the most basic queries need extensive programming knowledge, and integrated BI tools are insufficient.

Performance: Consider the same system as before in terms of performance (RPOs and RTOs). Because it is impossible to parallelize database file processing beyond a certain point, it will be impossible to achieve customer SLAs in terms of RTO if per row processing is sluggish. Due to the same parallelization limitations, recovery procedures are also impacted. This means that backup and recovery processing for 100 billion rows over 10 million files must be quick, and a solution must dramatically minimize per-row and per-file database file processing.

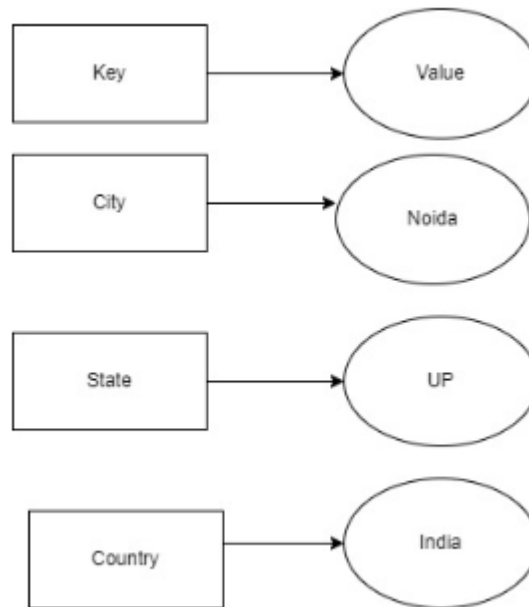
Data Integrity: Verifying data integrity at the block level is another issue with distributed NoSQL databases. Checksum work for scale-up databases because the restored data is physically identical to the backup data. The restored data in scale-out databases is semantically comparable to the backup data, but it is not physically identical. In this situation, we'll need to come up with a unique method for identifying semantic equivalence between recovered and backup data, which will allow us to spot data corruption issues that may arise throughout the backup and restoration process.

Expertise: Despite the fact that there are millions of developers throughout the world, the majority of them are familiar with RDBMS architecture. All NoSQL developers should be considered to be in the early stages of their careers and have yet to achieve expert status. While this problem will eventually be resolved, selecting the correct developer is now difficult.

De-duplication: In traditional database systems, de-duplication is simple—a block with identical bitwise values. Data copies are not always identical in modern NoSQL database storage systems because the sequencing and flushing of changes vary between nodes, creating a new challenge for de-duplication.

Key-Value Data Model in NoSQL:-

- A key-value data model or database is also referred to as a key-value store. It is a non-relational type of database.
- In this, an associative array is used as a basic database in which an individual key is linked with just one value in a collection.
- For the values, keys are special identifiers. Any kind of entity can be valued. The collection of key-value pairs stored on separate records is called key-value databases and they do not have an already defined structure.



Some examples of key-value databases:

Here are some popular key-value databases which are widely used:

- **Couchbase:** It permits SQL-style querying and searching for text.
- **Amazon DynamoDB:** The key-value database which is mostly used is Amazon DynamoDB as it is a trusted database used by a large number of users. It can easily handle a large number of requests every day and it also provides various security options.
- **Riak:** It is the database used to develop applications.
- **Aerospike:** It is an open-source and real-time database working with billions of exchanges.
- **Berkeley DB:** It is a high-performance and open-source database providing scalability.

Document Data Model: -

- A Document Data Model is a lot different than other data models because it stores data in JSON, BSON, or XML documents. In this data model, we can move documents under one document and apart from this, any particular elements can be indexed to run queries faster.
- Often documents are stored and retrieved in such a way that it becomes close to the data objects which are used in many applications which means very less translations are required to use data in applications. JSON is a native language that is often used to store and query data too.

So in the document data model, each document has a key-value pair below is an example for the same.

```
{  
  "Name" : "Yashodhra",  
  "Address" : "Near Patel Nagar",  
  "Email" : "yahoo123@yahoo.com",
```

```
"Contact" : "12345"
```

```
}
```

Examples of Document Data Models :

- Amazon DocumentDB
- MongoDB
- Cosmos DB
- ArangoDB
- Couchbase Server
- CouchDB

Applications of Document Data Model

- **Content Management:** These data models are very much used in creating various video streaming platforms, blogs, and similar services. Because each is stored as a single document and the database here is much easier to maintain as the service evolves over time.
- **Book Database:** These are very much useful in making book databases because as we know this data model lets us nest.
- **Catalog:** When it comes to storing and reading catalog files, these data models are very much used because it has a fast reading ability if in case catalogs have thousands of attributes stored.
- **Analytics Platform:** These data models are very much used in the Analytics Platform.

column store database:-

- A column store database is a type of database that stores data using a column-oriented model.

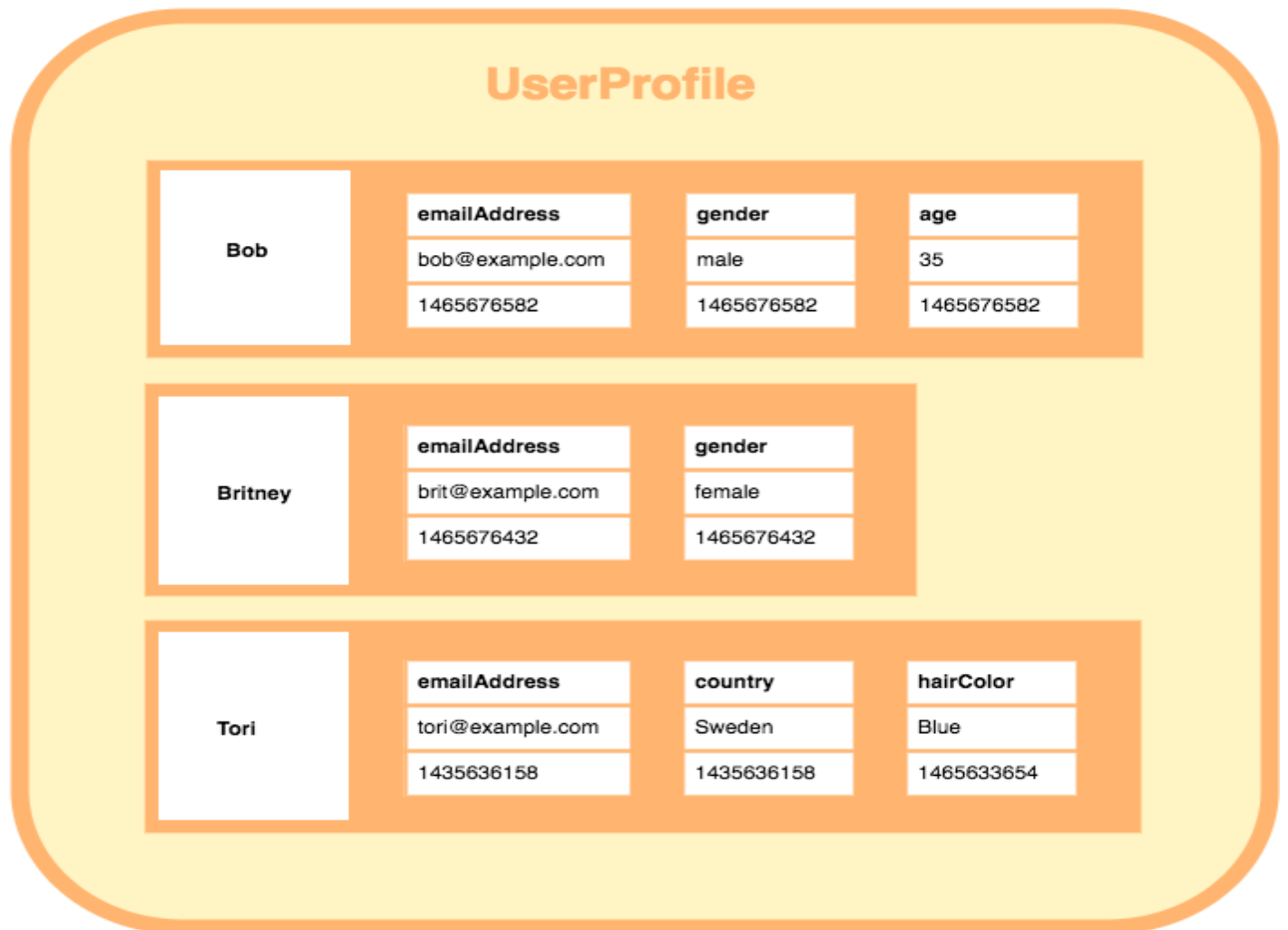
A column store database can also be referred to as a

- Column database
- Column family database
- Column-oriented database
- Wide column store database
- Wide column store
- Columnar database
- Columnar store

The Structure of a Column Store Database

Column store databases use a concept called a *keyspace*. A keyspace is kind of like a [schema](#) in the relational model. The keyspace contains all the column

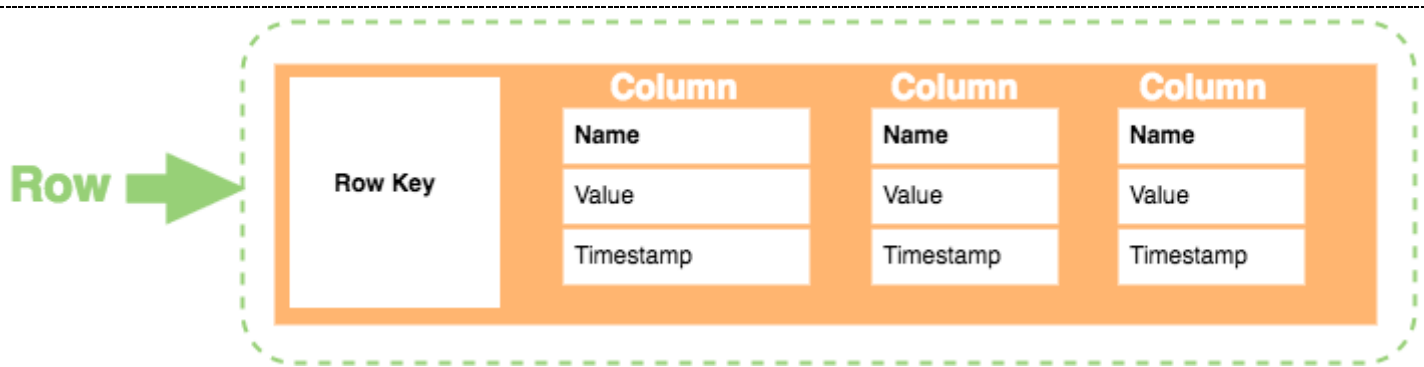
families (kind of like [tables](#) in the relational model), which contain rows, which contain columns.



As the above diagram shows:

- A **column family** consists of multiple rows.
- Each **row** can contain a different number of columns to the other rows. And the columns don't have to match the columns in the other rows (i.e. they can have different column names, data types, etc).
- Each **column** is contained to its row. It doesn't span all rows like in a relational database. Each column contains a name/value pair, along with a timestamp

Here's how each row is constructed:



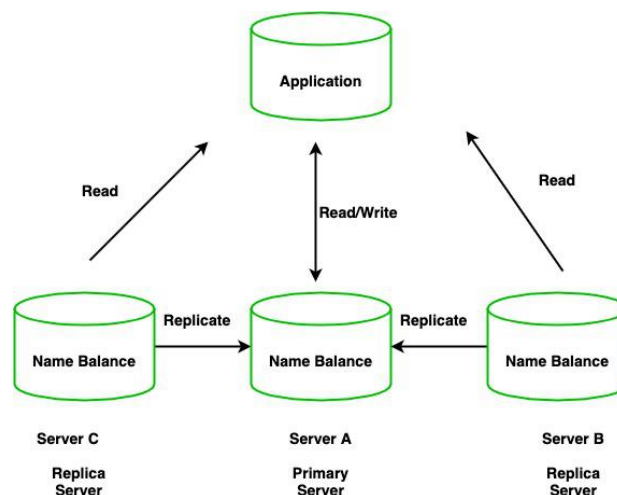
Replication and Sharding:-

Replication can be simply understood as the duplication of the data-set whereas sharding is partitioning the data-set into discrete parts.

By sharding, you divided your collection into different parts. Replicating your database means you make imagers of your data-set. In terms of functionality delivered.

Replication: -

Replication is the method of duplication of data across multiple servers. For example, we have an application and it reads and writes data to a database and says this server A has a name and balance which will be copied/replicate to two other servers in two different locations.



By doing this, will get redundancy and increases data availability with multiple copies of data on different database servers. So, it will increase the performance of reading scaling. The set of servers that maintain the same copy of data is known as replica servers or MongoDB instances.

Replication Key Features :

- Replica sets are the clusters of N different nodes that maintain the same copy of the data set.
- The primary server receives all write operations and record all the changes to the data i.e, oplog.
- The secondary members then copy and apply these changes in an asynchronous process.
- All the secondary nodes are connected with the primary nodes. there is one heartbeat signal from the primary nodes. If the primary server goes down an eligible secondary will hold the new primary.

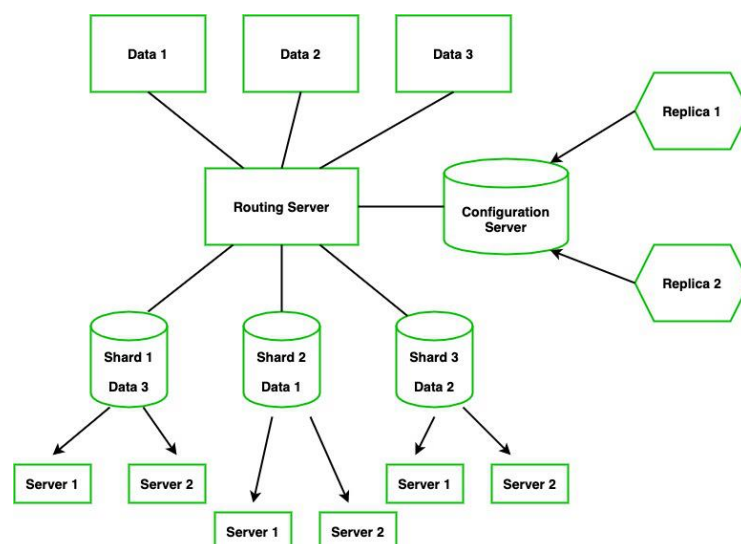
Sharding:-

Sharding is a method for allocating data across multiple machines. MongoDB used sharding to help deployment with very big data sets and large throughput the operation. By sharding, you combine more devices to carry data extension and the needs of read and write operations.

Why Sharding?

- Database systems having big data sets or high throughput requests can doubt the ability of a single server.
- For example, High query flows can drain the CPU limit of the server.
- The working set sizes are larger than the system's RAM to stress the I/O capacity of the disk drive.

How does Sharding work?



Now, instead of one signal as primary, we have multiple servers called Shard. We have different routing servers that will route data to the shard servers. For example: Let say we have Data 1, Data 2, and Data 3 this will be going to the routing server which will route the data (i.e, Different Data will go to a particular Shard) Each Shard holds some pieces of data. Here the configuration server will hold the metadata and it will configure the routing server to integrate the particular data to a

shard however configure server is the MongoDB instance if it goes down then the entire server will go down, So it again has Replica Configure database.

Introduction to MongoDB

Here's the background on MongoDB:

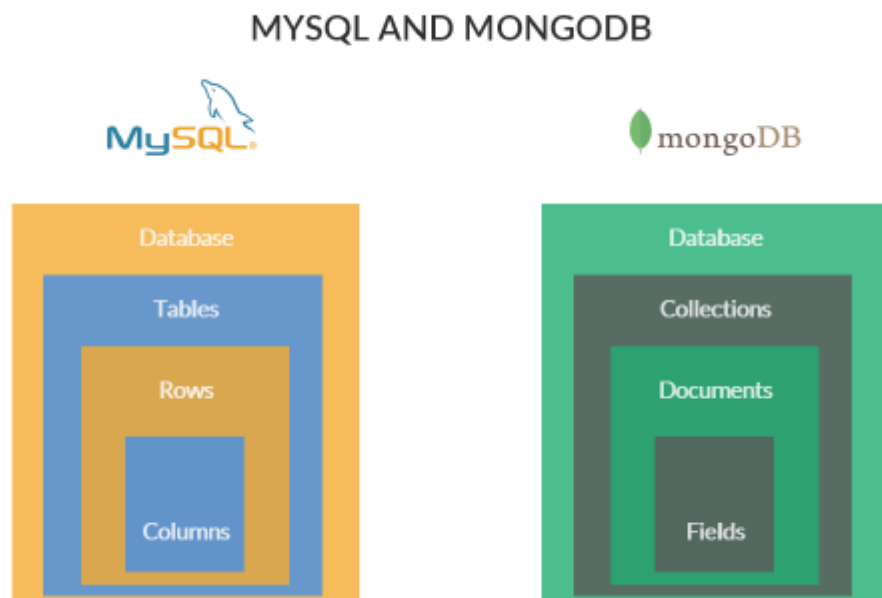
- It is an open-source, document-based NoSQL database.
- It was created in 2007 to build databases that can support humongous amounts of data to ensure scalability.
- It provides high performance, scalability, and data modeling features.

Some of the key features of mongoDB include:

- It is a document-based datastore.
- MongoDB is a schema-less database and it stores data in the form of JSON-like documents.
- MongoDB provides immediate consistency.
- It is used in mobile applications, IoT applications, content management systems, etc.
- Key customers of MongoDB include Google. CISCO, Adobe, Royal Bank of Scotland, etc.

Structure of MongoDB

The databases in MongoDB store all the information in the form of collections. These collections act as tables in SQL and store all the information in them in the form of documents. Here's an illustration that will help you understand the similarities and differences between SQL and MongoDB better.



The documents store the information in MongoDB in the form of key:value pairs like 'name': 'John' or 'Price':1000. An example of how documents are stored in MongoDB is shown below.

```
{
    "_id" : ObjectId("5ef58d5d1564fba7f8bea373"),
    "Order_ID" : "TEC-103",
    "Name" : "Plantronics",
    "Sales" : 2400,
    "Profit" : 760
}
```

The following operations are useful in creating new collections and recording data in MongoDB.

- use <database> - This command is used to switch to the database mentioned or create a new database with the name as passed in the command.
- db.createCollection('<collection name>') - Creates a new collection with the given name
- db.collection.insert() - Inserts a document into the collection
- db.collection.find() - Finds all the documents present in the given collection

CRUD Operation

Every database management system has a series of 4 essential steps that it needs to perform. These steps are abbreviated as CRUD operations or Create, Read, Update and Delete operations. These operations help us to store, retrieve, update and remove records from any type of database system that we're using.

The CRUD operations in MongoDB are given by the following commands.

- Create a new document using the db.collection.insert():

You can create or select a database, choose the collection and then insert the document by passing the key-value pairs inside the bracket.

For example, if you're adding a new document to a collection 'blogs', the syntax would be db.blogs.insert({'blog_id': 101,'likes':24,'comments':12})

- Read the documents using db.collection.find():

- All the documents in the collection will be returned.
- Can be used to filter documents based on a specific field and value pair.
- Also, additional conditional statements and logical operators can also be added to further filter the documents.

- Update the values using db.collection.update():

- Helps you find the document that you need to update
- Then mention all the details with which you need to update the values of the document.

For example, In the above case if we want to update the number of likes to 30 and the number of comments to 14 then you need to perform the following operation:

db.blogs.update({'blog_id': 101},{ '\$set': {'likes':30, 'comments':14}})

- Delete a document using db.collection.remove():

- Helps you match the document that you want to delete and then delete it with the db.collection.remove command.

Data Types in MongoDB

These are the data types that are possible in MongoDB.

- String: This is the most used data type in MongoDB.
- Integer: It is used to store a numerical value.
- Boolean: It stores a Boolean (true/false) value.
- Double: It is used to store floating-point values.
- Object: It is used for embedded documents.
- Arrays: It stores a list of multiple values into one key.
- Date: It stores the current date or time in UNIX time format.

Querying in MongoDB

In order to query the required results in MongoDB we first start by importing the required dataset. This is done using the mongoimport utility that needs to be installed on your device. (See the optional session in the module for installation instructions).

The command for importing the data into MongoDB would be as follows:

```
mongoimport --db <database name> --collection <collection name> --type csv
--file <filepath> --headerline
```

Comparison Operators

Comparison operators are used to compare strings or numerical values to perform evaluations. These help us filter the output of queries to only retrieve the most relevant results. The comparison operators available in MongoDB are as follows:

- Equal to (\$eq).
- Greater than (\$gt).
- Greater than or equal to (\$gte).
- Less than (\$lt).
- Less than or equal to (\$lte).

Not equal to (\$neq).

- In (\$in).

The following table illustrates the instances where each of these operators can be used.

Operator	Syntax	Example	Command
Equal to	\$eq	Query the list of products manufactured by 'Toyota Group'	db.collection.find({'Manufactured By':{'\$eq: 'Toyota Group'}})
Greater than	\$gt	Query the list of products whose price is greater than \$299.99	db.collection.find({'Price':{'\$gt: 299.99}})
Greater than or equal to	\$gte	Query the list of products whose price greater than or equal to \$299.99	db.collection.find({'Price':{'\$gte : 299.99}})
Less than	\$lt	Query the list of products whose price is less than \$299.99	db.collection.find({'Price':{'\$lt: 299.99}})
Less than or equal to	\$lte	Query the list of products whose price is less than or equal to \$299.99	db.collection.find({'Price':{'\$lte : 299.99}})
Not equal to	\$neq	Query the list of products which are not 'in stock'	db.collection.find({'Stock-Status':{'\$neq: 'in stock'}})
In	\$in	Query the list of products which are either ['Packaged', Ready for transit', 'In-transit']	db.collection.find({'Status':{'\$in : ['Packaged', Ready for transit', 'In-transit']}})

Ac

Logical Operators

Logical operators are used when you want to compound two or more comparison operators. In logical operators, you learned the following

- AND
- OR
- NOT

The following table illustrates the instances where each of these operators can be used.

Operator	Syntax	Example	Command
AND	\$and	Query the list of products manufactured by 'Toyota Group' and is 'out-of-stock'	<code>db.collection.find(\$and: [{ 'Manufactured By': { \$eq: 'Toyota Group' } }, { 'Stock Status': { \$eq: 'out-of-stock' } }])</code>
OR	\$or	Query the list of products manufactured by 'Toyota Group' or whose country of origin is 'South Korea'	<code>db.collection.find(\$or: [{ 'Manufactured By': { \$eq: 'Toyota Group' } }, { 'Country of Origin': { \$eq: 'South Korea' } }])</code>
NOT	\$not	Query the list of products that are not manufactured by 'Toyota Group'	<code>db.collection.find({ 'Manufactured By': { \$not: { \$eq: 'Toyota Group' } } })</code>

Indexing

Indexes in MongoDB play a crucial role in enhancing the performance of database operations. By creating indexes on specific fields, MongoDB can quickly locate data without scanning every document in a collection and speeding up query execution.

In this article, We will learn about the MongoDB Index and its various Index Properties with the help of examples to discover how they can optimize our database operations.

What is the MongoDB Index?

In MongoDB, indexes are data structures that store a small portion of the collection's data in an easy-to-traverse form. They make the database perform rapid searches by using indexed fields rather than downloading whole documents. Indices resolve issues on complex queries more efficiently.

MongoDB has different types of indexing mechanisms like single field indexes, compound indexes, multikey indexes, geospatial indexes, and more. Indexes perform specific tasks for creating databases that meet the performance needs of developers' applications to their specific departments.

Types of MongoDB Indexes

Single Field Index

Compound Index

Multikey Index

Geospatial Index

Text Index

Index-Properties

Index Properties

1. Unique Indexes

Unique indexes ensure that the fields have unique values within the collection for all the documents. Such feature guarantees the unique value per each one of the collection's documents. Entering or altering a document with duplicate value will result in an error. Unique indexes can be used for data integrity constraints like ensuring that the information is unique and for preventing duplicated entries.

To make an index unique, we can use the `db.collection.createIndex()` method and set the unique option to true.

```
db.collection.createIndex({ "fieldName": 1 }, { unique: true })
```

Example:

Let's create a MongoDB collection named employees and ensure that the name field is unique for each document by creating a unique index:

Create a unique index on the "name" field

```
db.employees.createIndex({ "name": 1 }, { unique: true })
```

Now, let's insert a document into the employees collection:

```
db.employees.insertOne({ "name": "John Doe", "position": "Manager", "department": "Sales" })
```

Output:

UniqueIndex

Unique Index

2. Partial Indexes

Partial indexes focus on indexing only a subset of documents in a collection, specifically those that meet a predefined filter expression. By indexing a smaller set of documents, partial indexes reduce the overall index size compared to indexing the entire collection. This can lead to more efficient use of storage and memory.

Partial indexes are useful when we want to index data based on certain conditions or criteria, allowing for more targeted and efficient indexing strategies.

To create a partial index, use the `db.collection.createIndex()` method with the `partialFilterExpression` option.

```
db.collection.createIndex({ "fieldName": 1 }, { partialFilterExpression: { "status": "active" } })
```

Example:

Suppose we want to create a partial index on the department field for documents where the position field is set to "Manager". Let's create this partial index. Create a partial index on the "department" field for documents with "position" set to "Manager".

```
db.employees.createIndex(  
  { "department": 1 },  
  { partialFilterExpression: { "position": "Manager" } })
```

Output:

PartialIndex

Partial Index

Explanation: The output `department_1` indicates that the partial index on the department field has been successfully created with the index key `department_1`.

3. Sparse Indexes

Sparse indexing indexes only documents that have the indexed field, excluding those that do not have it. It is beneficial when indexing fields that are missing or sparse in most documents. Sparse indexes help optimize index size and query performance by indexing only the documents containing the indexed element.

To create a sparse index, use the `db.collection.createIndex()` method with the `sparse` option set to `true`.

```
db.collection.createIndex({ "fieldName": 1 }, { sparse: true })
```

Example:

Suppose we want to create a sparse index on the position field to include only documents that have the position field populated. Let's create this sparse index. Create a sparse index on the "position" field.

```
db.employees.createIndex({ "position": 1 }, { sparse: true })
```

Output:

SparseIndex

Sparse Index

Explanation: The output position_1 indicates that the index on the position field has been successfully created.

4. TTL Indexes

TTL indexes are special indexes in MongoDB used for automatic removal of documents from a collection after a specified time. They are ideal for implementing data expiration policies, like clearing temporary or cached data. Common use cases include managing time-sensitive data such as session information or log entries.

To set up a TTL index, apply the `createIndex()` method to a field that holds either a date or an array of date values. Specify the `expireAfterSeconds` option with your chosen TTL value in seconds.

```
db.collection.createIndex({ "createdAt": 1 }, { expireAfterSeconds: 3600 })
```

Example:

Let's create a TTL index on the `createdAt` field to automatically delete documents after 24 hours:

```
db.employees.createIndex({ "createdAt": 1 }, { expireAfterSeconds: 86400 })
```

Output:

TTLIndex

TTL Index

Explanation: The output `createdAt_1` indicates that the index on the `createdAt` field has been successfully created.

Best Practices for Index Utilization

To leverage index properties effectively and optimize database performance in MongoDB, developers should adhere to best practices:

Analyze Query Patterns: Understand the application querying patterns and focus on the frequently executed queries that will better benefit from indexing.

Create Indexes Strategically: Create indexes that match query patterns. It should be you're prioritizing fields involved in filter condition, sorting and aggregation.

Monitor Index Usage: Watch usage of indexes and to check if they are well performing, know bottlenecks and areas of optimization.

Avoid Over-Indexing: On the one hand, indexing enhances speed of queries, while on the other, too many indexes increase storage overhead and make write operations slower. It is recommended to index selectively basing on actual usage profiles to prevent information overload.

Regular Maintenance: Indexes need to be reviewed and amended from time to time depending on application requirements evolution and information governance strategy.